

2021年度（2021年10月・2022年4月入学）入学試験問題 解答 ― 大問 3

千葉大学大学院融合理工学府 数学情報科学専攻 情報科学コース 専門科目／C 言語（二分木の走査・一致判定）

問 1 配列 n のようなデータ構造の名称

二分木 (binary tree)

n[0] (val=0) を根として、左の子 n[1] (val=1)、右の子 n[4] (val=4)、さらに n[1] の左の子 n[2] (val=2)、右の子 n[3] (val=3) がつながっており、各ノードが高々 2 つの子 (lt, rt) を持つ木構造である。

問 2 関数 traverse の空欄

空欄	解答
Ⓐ	<code>n == NULL</code>
Ⓑ	<code>traverse(n->lt);</code>
Ⓒ	<code>traverse(n->rt);</code>

完成したコード：

```
void traverse(node* n) {
    if (n == NULL)
        return;

    printf("%d ", n->val);
    traverse(n->lt);
    traverse(n->rt);

    return;
}
```

NULL（葉の先）に到達したら戻り、自分の値を表示してから左部分木・右部分木を再帰的に走査する（行きがけ順、preorder の深さ優先探索）。木の各ノードはちょうど 1 回ずつ訪問されるので、重複なく列挙される。

問 3 関数 dfs_match の空欄

空欄	解答
Ⓓ	<code>return 1;</code> (片方だけ NULL → 不一致)
Ⓔ	<code>return 0;</code> (両方 NULL → この部分は一致)
Ⓕ	<code>return 1;</code> (n1 ≠ NULL かつ n2 = NULL → 不一致)
Ⓖ	<code>n1->val != n2->val</code>
Ⓕ	<code>dfs_match(n1->lt, n2->lt) == 1</code>

①	<code>dfs_match(n1->rt, n2->rt) == 1</code>
---	---

完成したコード：

```
int dfs_match(node* n1, node* n2) {
    if (n1 == NULL) {
        if (n2 != NULL) {
            return 1;
        }
        return 0;
    }
    if (n2 == NULL)
        return 1;

    if (n1->val != n2->val)
        return 1;
    if (dfs_match(n1->lt, n2->lt) == 1)
        return 1;
    if (dfs_match(n1->rt, n2->rt) == 1)
        return 1;

    return 0;
}
```

Ⓜ・①は `dfs_match(n1->lt, n2->lt) != 0` や単に `dfs_match(n1->lt, n2->lt)` でも同じ意味。なお問題の冊子では戻り値型が `void` と書かれているが、値を `return` しているので本来は `int` が正しい（問題の誤植と考えられる）。

問 4 関数 `bfs_match` の空欄

`np1`, `np2` をキュー（待ち行列）として使い、比較すべきノードの組を `head`（取り出し位置）と `tail`（追加位置）で管理しながら幅優先で比較する。

空欄	解答
ⓐ	<code>np1[tail] = n1;</code>
ⓑ	<code>np2[tail++] = n2;</code> (= <code>np2[tail] = n2; tail++;</code>)
ⓒ	<code>head < tail</code> (または <code>head != tail</code> : キューが空でない間)
ⓓ	<code>n1 = np1[head];</code>
ⓔ	<code>n2 = np2[head++];</code> (= <code>n2 = np2[head]; head++;</code>)
ⓕ	<code>return 1;</code>
ⓖ	<code>return 1;</code>
ⓗ	<code>n1->val == n2->val</code>
ⓓ	<code>np1[tail] = n1->lt; np2[tail] = n2->lt; tail++;</code> <code>np1[tail] = n1->rt; np2[tail] = n2->rt; tail++;</code> (左右の子の組をキューに追加)
ⓔ	<code>return 1;</code> (値が異なる → 不一致)

完成したコード：

```
int bfs_match(node* n1, node* n2) {
    node* np1[PLENTY];
    node* np2[PLENTY];
    int head = 0;
    int tail = 0;

    np1[tail] = n1;
    np2[tail++] = n2;

    while (head < tail) {
        n1 = np1[head];
        n2 = np2[head++];

        if (n1 != NULL && n2 == NULL)
            return 1;
        if (n1 == NULL && n2 != NULL)
            return 1;

        if (n1 != NULL && n2 != NULL) {
            if (n1->val == n2->val) {
                np1[tail] = n1->lt;
                np2[tail] = n2->lt;
                tail++;
                np1[tail] = n1->rt;
                np2[tail] = n2->rt;
                tail++;
            } else {
                return 1;
            }
        }
    }
    return 0;
}
```

動作：まず根の組 (n1, n2) をキューに入れる。キューから 1 組ずつ取り出し、(i) 片方だけ NULL なら形が違うので 1 を返す、(ii) 両方 NULL なら何もしない（その先はない）、(iii) 両方非 NULL なら値を比較し、等しければ左の子の組と右の子の組をキューの末尾に追加、異なれば 1 を返す。キューが空になるまで不一致が見つからなければ完全一致であり 0 を返す。dfs_match の再帰（スタック）をキューに置き換えたことで、根に近い階層から順（幅優先）に比較される。

問 3 と同様、戻り値型は本来 int が正しい（冊子では void と誤植）。㉔と㉕、㉖と㉗の tail++ / head++ の置き場所は、2 つの空欄のどちらに含めてもよい。